Notification (Kafka → email)

1. Feature Overview

After successful account **writes**, BankOne publishes a `BankActionEvent` to Kafka. A consumer builds a transactional HTML+plain email and delivers it to the **customer's email** (from customer create), with `MAIL_NOTIFY_TO` as fallback.

Status: Implemented for open account, deposit, withdraw, and transfer (destination customer notified on transfer). Local: Docker Kafka + Mailpit. Render: Aiven Kafka + Twilio SendGrid **HTTPS** API (`MAIL_TRANSPORT=sendgrid`).

2. Business Purpose

Notify customers of ledger-changing activity without blocking the HTTP request on mail SMTP. Kafka decouples write path from delivery.

3. User Workflow

1. Staff opens account / deposits / withdraws / transfers in UI
2. API saves Postgres, then publishes to topic `bankone.notifications`
3. Consumer sends email
4. Local: view inbox at `http://localhost:8025` (Mailpit)
5. Render: customer receives mail via SendGrid (check Spam until domain auth)

4. Execution Flow

```
AccountServiceImpl (after save)
  → NotificationEventPublisher.publish(...)
  → Kafka topic bankone.notifications
  → NotificationEventConsumer.onMessage
  → NotificationEmailComposer.compose
  → NotificationMailService.send (SMTP or SendGrid)
```

Fail-soft: Kafka/mail failures are logged; business transaction is not rolled back.

5. Database Tables

None dedicated. Recipient comes from `customers.email` via account → customer.

6. REST APIs

No dedicated notification REST API. Side effect of account write APIs.

7. Controllers

None.

8. Services / classes (`com.bankone.notification`)

Class	Role
`NotificationEventPublisher`	Kafka produce (`BankActionEvent`)
`NotificationEventConsumer`	`@KafkaListener` → compose → send
`BankActionEvent`	Payload (incl. `recipientEmail`)
`NotificationEmailComposer`	Subject + plain + HTML template
`NotificationEmailContent`	Record (subject, plain, html)
`NotificationMailService`	Send interface
`SmtpNotificationMailService`	Default (Mailpit / SMTP)
`SendGridNotificationMailService`	`@Primary` when transport=sendgrid

9. Events published (`AccountServiceImpl`)

Action	When	Recipient
`ACCOUNT_OPENED`	After open account save	Customer email
`DEPOSIT_COMPLETE`	After deposit	Customer email
`WITHDRAW_COMPLETE`	After withdraw	Customer email
`TRANSFER_SUCCESS`	After transfer	Destination customer email

Summaries must use `customerLabel(Account)` / field getters — never concatenate `Customer` entity (`Customer@hash` smell).

10. Configuration

See `.env.example` and DEPLOY.md.

Local defaults: Kafka `localhost:9092`, Mailpit SMTP `localhost:1025`.

11. Future Extension Guide

- Notify **both** transfer parties
- Outbox pattern for DB+Kafka atomicity
- Customer create / status-change events
- Domain authentication (SPF/DKIM) on SendGrid to reduce Spam

Future Modification Guide

Change	Files / classes	Impact
New event type	`AccountServiceImpl` publish + `NotificationEmailComposer` titles	Consumer auto-sends
Change mail provider	New `NotificationMailService` + `@ConditionalOnProperty`	Render/local env
Topic rename	`app.kafka.notification-topic` / Aiven topic	Redeploy both sides